

What each model is, how it learns, and what it costs

Capacity first, then the detail. Four models: two are trained once and shipped to everybody. One is trained separately for every single user, every week, in two seconds. One we rent and never train. This is what each one does, why it was chosen over the alternatives, how it adapts as a user changes, where it runs in production, and what the bill looks like at a thousand, a hundred thousand and a million users.

CAPACITY

How much load these models carry

Speeds measured on one laptop CPU core, turned into capacity. One "user" here means 100 emails a day.

One 4-core server, today 9.4M emails a day · **26,800 users**. That is a \$24/month box.
Same server, after quantisation 27.4M emails a day · **78,400 users**. One afternoon's work; the script is written.
Memory for all three models 2.25 GB Measured, everything loaded at once. Not estimated.
Reaches a model at all 20% Free rules drop the rest before any model runs. This is why the numbers are small.

MODEL	RUNS ON	SPEED	EMAILS / SEC ONE CORE	EMAILS / DAY ONE CORE	USERS PER CORE
1 Reader xlm-roberta-base	20% of mail	110 ms	9	785,455	39,273
quantised (ONNX int8)	20%	25 ms	40	3,456,000	172,800
2 Decider LightGBM · 300 trees	20% of mail	1 ms	1,000	86,400,000	4,320,000
3 Name finder xlm-roberta-base	8% of mail	120 ms	8	720,000	90,000
quantised (ONNX int8)	8%	30 ms	33	2,880,000	360,000
4 Writer gemini-flash-latest	8% of mail	~2 s	Not our hardware. Capacity is the vendor's rate limit — and it is the only thing that costs money per call.		

SERVERS NEEDED — 4 vCPU / 8 GB EACH, QUANTISED

MEMORY — MEASURED, NOT ESTIMATED

USERS	EMAILS / DAY	SERVERS
1,000	100,000	1
10,000	1,000,000	1
100,000	10,000,000	2
1,000,000	100,000,000	13

As built rather than quantised, multiply by 3. No GPU at any size.

python alone	18
MB	
+ torch	196
MB	
+ model 1 (reader)	1,707
MB	
+ model 3 (name finder)	2,293
MB	
+ model 2 (decider)	2,298
MB	
everything loaded	2.25
GB	

Which model is the constraint, and which never will be

The decider will never be a bottleneck. One core carries **4.3 million users** of it — because it reads 17 numbers, not text. The model that personalises the whole system is effectively free to run and free to retrain.

The reader is the bottleneck, at 39,273 users per core. Which makes quantising it the single obvious optimisation: **4× the capacity for one afternoon's work**, and the conversion script is already written and installed.

What these numbers are, and what they are not

Per-email speeds and memory are **measured on real hardware**. Everything derived from them — users per core, servers needed — is **calculated** from stated assumptions: 100 emails per user per day, 20% surviving the free rules to reach a model, 8% reaching the drafting step, and a 3.5× factor for business-hours peaks. **None of it has been load-tested under real concurrency.** Believing a spreadsheet about throughput is how projects get surprised.

THE QUESTIONS ASKED

Answered directly, before anything else

ASKED

Jaise jaise naya user aayega toh hume dobara isko fine tune karna hai ya ye as it is chalta rahega? Ye model continuous learning ya re-adaptive learning hai? Model apne aap ko learn karta rehta hai kya? Jaise jaise user ka writing style update ho raha hai toh wo apne aap ko update kar raha hai ke nahi?

Q1 A new user signs up. Do we fine-tune again?

Three of the four models: no. Nothing happens at all.

The reader and the name finder are trained once and shipped as fixed files. A new user downloads the same weights every other user has. Signing up costs **zero training** and zero GPU time. The writer is rented, so it is not ours to train either.

One model is built fresh for them: the decider.

It is trained on that person's own mailbox history — who they open, who they answer, who they ignore. It takes **about two seconds**. That is not a rounding error in our favour; it is the reason personalisation is affordable at all, and it is why that model is 300 decision trees rather than a neural network.

Q2 Is it continuous learning? Does the model keep learning by itself?

The system adapts continuously. The neural networks deliberately do not.

That distinction is the whole design, and it is a choice we made on purpose rather than a limitation. There are three separate adaptation loops running at three different speeds, and the fast ones are the ones that matter to a user. Detail in the next section.

What we do **not** do is let a live model update its own weights from incoming mail. Four reasons, all of them serious: a model that learns continuously forgets what it knew before; you cannot reproduce or explain a decision made last Tuesday; you cannot roll it back when it goes wrong; and **anyone who can send you an email could shape your model by sending you patterns**. That last one converts a spam problem into a security problem.

Q3 As the user's writing style changes, does it update itself?

Yes — immediately, and without retraining anything.

This is the part that usually surprises people. The user's writing style is **not stored in any model's weights**. It is stored as their actual sent emails, kept as vectors, plus a measured profile: how they greet, how they sign off, typical length, how formal, whether they use contractions.

When they write a new email, it joins the store. The next draft retrieves the four or five closest past replies — and those now include the new ones. **A style change takes effect on the very next email**, not after a retraining cycle, because there is no cycle. Nothing was ever baked in.

A second consequence, which matters commercially: because nothing about the user is inside any model's weights, **deleting their account genuinely deletes them**. There is no model to unlearn. That is a very hard promise to make once you have fine-tuned on someone.

Three loops, three speeds

We separated what has to change fast from what has to stay stable.
Fast things are cheap and reversible. Stable things are versioned and tested.

INSTANT · EVERY EMAIL

Writing style

Every email the user sends joins their voice store, and their profile is recomputed. The next draft uses it.

- **What changes:** stored examples and measured statistics
- **Cost:** a few milliseconds per sent email
- **Reversible:** delete a row
- **Not a model.** No weights involved anywhere.

WEEKLY · PER PERSON

Who matters to you

The decider is retrained from scratch on that person's fresh behaviour — who they opened and answered this week.

- **What changes:** 300 decision trees, that user's alone
- **Cost:** ~2 seconds of CPU
- **Reversible:** keep last week's file
- **This is real per-person learning,** and it is genuinely cheap.

MONTHS · EVERYBODY

Understanding language

The reader improves in versioned releases when we add labelled mail — including corrections users made to our answers.

- **What changes:** one shared model file, version-numbered
- **Cost:** ~4 GPU-minutes, once, for all users
- **Reversible:** ship the previous version
- **Tested before release.** Nobody gets an untested model.

NEVER — ON PURPOSE

Live weight updates

No model updates its own weights from incoming mail. This is a decision, not a gap.

- **Forgetting:** continual updates erase earlier knowledge
- **No explanation:** cannot reproduce last Tuesday's decision
- **No rollback:** a bad week is permanent
- **Poisoning:** anyone who can email you could shape your model

The way to say this in one sentence

"The system learns continuously; the neural networks are versioned. Personal behaviour and writing style adapt in seconds because they live in data, not in weights — and language understanding stays stable because we ship it as a tested release, not as something that drifts."

WHY THE DECIDER IS TREES AND NOT A NEURAL NETWORK

The number that settles the architecture

"Trained separately for every user, every week" is either trivially cheap or completely impossible, depending entirely on this one choice.

Retraining a personal model for every user, every week:

100,000 users

300 decision trees each	0.33 CPU cores, sustained
a 278M encoder each	40 GPUs, running continuously

1,000,000 users

300 decision trees each	3.31 CPU cores, sustained
a 278M encoder each	397 GPUs, running continuously

Three CPU cores against four hundred GPUs. That is not an optimisation; it is the difference between a feature that exists and one that does not.

The decider only ever handles numbers — seventeen of them — which is exactly what gradient-boosted trees are best at. It never reads text, because stage 8 already did that. So the architecture choice and the personalisation promise turn out to be the same decision, made once.

Calculated from 2 seconds per user per week for trees, and 4 GPU-minutes per user per week for a 278M encoder. Assumptions stated so they can be argued with.

MODEL BY MODEL

Every model, in full

1 • The reader

OURS • TRAINED ONCE FOR EVERYONE

[xlm-roberta-base](#) → [XLMRobertaForSequenceClassification](#)

WHAT IT DOES

Reads the cleaned email text and answers one question: **is this person asking me for something, or just telling me something?** It does not summarise, decide or write.

PARAMETERS

278,045,186 · 12 layers · hidden 768 · 12 attention heads · vocab 250,002 · max 514 tokens · 2 output labels

HOW IT WAS TRAINED

Fine-tuned on **199 emails read and labelled by hand** from a real mailbox. Class-weighted, because the classes were 56 against 143. Free Kaggle T4 GPU, **~4 minutes**.

SPEED & RESOURCES

~110 ms on one CPU core. **~25 ms after ONNX + int8**, roughly 4× smaller too. No GPU needed to run it, ever.

WHERE IN THE PIPELINE

Stage 8. The first model any email meets, and only after roughly 80% of mail has already left on free rules.

OF WHICH

192M (69%) is the vocabulary table — 250,002 word-pieces across a hundred languages. That table is why one model covers US, Canada and UK, French included.

PER NEW USER

Nothing. The same file is shipped to everybody. Signing up costs no training at all.

MEASURED IMPACT

Confidence 0.82–0.90 on real mail. Started as a four-way classifier at 0.36–0.54; collapsing it to the binary question stage 9 actually asks turned 5-and-10-example classes into 56 vs 143.

WHY THIS MODEL, AND WHAT ELSE WAS CONSIDERED

OPTION	WHY NOT
mDeBERTa-v3-base	Tried first. Failed twice. Trained one epoch then went to NaN — and a broken model saves and loads without complaint, predicting one label for everything. Cost us a download and a puzzled ten minutes before we found it.
distilbert-base	English only. Canada needs French; the market is US, Canada, UK.
bert-base-multilingual	Older, measurably weaker on classification, same size. No reason to prefer it.
A hosted API (GPT / Gemini / Claude)	Would work well and charge for every email, forever . At 100k users that is 2 million classification calls a day. This is the cost the whole architecture exists to avoid.
Something under 66M	Measured cliff. Below 66M costs −8.7 F1 . Genuinely too small.
Something over 400M	Measured plateau. 33M→0.931, 44M→0.954, 184M→0.955, 434M→0.961. Ten times the parameters bought +0.007.
xlm-roberta-base	Chosen. MIT licence, free on Hugging Face, multilingual, stable under this training setup, and sits in the flat part of the curve just past the cliff.

In production

Loaded once per worker process and held in memory — **not** loaded per email, which would dominate the 25 ms. Runs as ONNX with int8 quantisation on ordinary CPU workers behind a job queue. No GPU in the serving path anywhere. Model files are version-numbered and shipped like any other release artefact, so a bad model is rolled back rather than repaired in place.

2 · The decider

OURS · TRAINED PER PERSON, WEEKLY

LightGBM 4.7.0 → [gate_model.txt](#)

WHAT IT DOES

Answers **"does this need *you*?"** — which is a different question from "is this a request". It never reads the email; it consumes what stage 8 already understood.

STRUCTURE

300 decision trees, voting. **No neural parameters at all**. 17 input features. 1,007 KB on disk.

HOW IT WAS TRAINED

On the user's own mailbox, with answers **the mailbox already knows** — did they open it, did they reply. Nobody labels anything by hand. Gmail recorded those answers over years of ordinary use.

SPEED & RESOURCES

~1 ms per email, one CPU core. Training all 1,000,000 users weekly: **3.3 CPU cores, sustained**.

WHERE IN THE PIPELINE

Stage 9. The last thing before anything costs money.

THE 17 FEATURES

sender total · open rate · recent · machine-generated · has unsubscribe · one-click · is a reply · auto-reply · in inbox · subject length · subject has "?" · size · hour · weekday · business hours · first contact · domain total

PER NEW USER

Trained fresh, in ~2 seconds, from their backfilled history. Retrained weekly from that week's behaviour.

MEASURED

AUC **0.684** on this mailbox. Live: **0.96** on a friend's reply, **0.00** on a job alert.

WHY THIS MODEL, AND WHAT ELSE WAS CONSIDERED

OPTION	WHY NOT
A neural network	Arithmetically impossible per person . 397 GPUs running continuously to retrain a million users weekly. Against 3.3 CPU cores for trees.
Logistic regression	Cannot capture interactions — and this problem is <i>all</i> interaction. "Machine-generated" means one thing from a stranger and another from a sender you open every day.
XGBoost	Comparable accuracy, honestly. LightGBM trains faster on small per-user tables, and per-user speed is the entire point here.
Reusing stage 8	Measured: the email's raw words predict replies at 0.511 — chance is 0.500. The history of who you answer predicts at 0.692 . Whether you reply is mostly about the relationship, not the sentence.
LightGBM	Chosen. Handles exactly the kind of input this is, trains in seconds, explains its own decisions, and makes weekly per-person retraining a background job nobody notices.

Honest, and worth saying out loud

0.684 is **below** the published ceiling of 0.72–0.79 for this task. Two reasons, both about data rather than design: the test account has almost no sent mail, so the strongest signal is nearly absent; and it is 95% notifications, so there is little variety to learn from. This is a data ceiling and it moves when the data moves.

It is also **blended** with hand-set weights rather than trusted alone — two mediocre estimates that disagree for different reasons beat either one, and a bad training week can then never swing a decision by itself.

3 · The name finder

OURS · TRAINED ONCE FOR EVERYONE

[xlm-roberta-base](#) → [XLMLobertaForTokenClassification](#)

WHAT IT DOES

Finds names, addresses and personal details in text so they can be replaced with placeholders before anything crosses to an outside company.

PARAMETERS

277,539,953 · same 12 layers, 768 hidden, 12 heads · **113 output labels**, collapsed by us to PERSON, EMAIL, PHONE, ADDRESS, DATE, ORG

PER NEW USER

Nothing. Identifiers do not change per person. Shipped as a fixed file, rarely updated.

MEASURED

0.963 on tokens, **0.922 on identifier tokens** — on that dataset.

WHERE IN THE PIPELINE

Stage 10 — **the only door out of the building.** If text did not pass through that one function, it never reached anyone.

HOW IT WAS TRAINED

On [ai4privacy/pii-masking-200k](#), a free public dataset of 200,000 texts with every identifier pre-marked. Kaggle T4, **~3 minutes. No mailbox involved — nothing of ours left the building to train it.**

SPEED & RESOURCES

~120 ms CPU, and only on the small fraction of mail that reaches drafting. ~30 ms under ONNX int8.

RUNS AFTER PATTERNS, NOT BEFORE

Regular expressions claim cards, IBANs, emails and phone numbers first. Our model found every identifier in testing but **typed several wrongly** — a credit card called an SSN. Letting it go first would let a bad label displace a pattern that was right.

WHY THIS MODEL, AND WHAT ELSE WAS CONSIDERED

OPTION	WHY NOT
Presidio alone	Excellent at structured identifiers and blind to names in free text, which is precisely the hard part. We use its <i>approach</i> — patterns first — and add a model behind it.
Piirinha	Self-reports 93.12. Scored 0.34 on real documents.
A hosted API	Would mean sending the un-redacted text to an outside company <i>in order to decide what to hide from outside companies</i> . Self-defeating.
Larger encoders	Measured plateau. 33M→0.931, 44M→0.954, 184M→0.955, 434M→0.961.
xlm-roberta-base	Chosen. Same base as the reader, so one architecture to maintain, one tokenizer, one set of deployment problems.

The limit that does not go away, and must be said

On a benchmark of *real* documents, a model self-reporting **98.82** scored **0.27**. Nothing tested beat a non-expert human at 0.77. Cards and IBANs are near perfect because they have checkable structure. **Names and free text are not.**

So the claim we can defend is: "*we strip identifiers, we minimise what crosses, and we contract for zero retention*" — **not** "names are removed". Anyone promising the second thing has not measured it.

4 • The writer

RENTED • NEVER TRAINED BY US

gemini-flash-latest

WHAT IT DOES

Writes the actual reply, in the user's voice. Also answers stage 8 when the router decides our own model was unsure.

WHAT IT SEES

Placeholders only — [PERSON_1], [EMAIL_2] — plus the user's writing profile and their own closest past replies.
Never a real name.

RELIABILITY

Retry with exponential backoff, then **four fallback models** in order. Free tiers are busy; failing a pipeline because a server was momentarily loaded is not acceptable.

WHERE IN THE PIPELINE

Stage 11, and stage 6 on escalation. **The only step that costs money.**

PARAMETERS

Not published by Google. We do not host it, so it does not affect our sizing.

HOW OFTEN IT RUNS

Roughly **8% of mail**. Everything upstream exists to keep that number small.

WHY RENT THIS ONE, WHEN WE TRAINED THE OTHER THREE

OPTION	ASSESSMENT
Train our own writer	Would cost more than years of API calls and produce something worse. Classification is a narrow question with a right answer; writing prose that sounds like one specific human is open-ended, and that is the one thing large models genuinely win at.
A small local writer (7B, self-hosted)	Possible, and worth revisiting at scale — it becomes cheaper than the API somewhere above a few million drafts a month. It needs GPUs in the serving path, which nothing else here does. Not yet justified.
Claude / GPT	Genuine candidates. The decision should be made by blind comparison on real drafts that our own users rate — not by any public benchmark. Currently Gemini Flash, chosen for a usable free tier during the POC.
Gemini Flash, rented	Chosen for now. Fast, cheap per call, and swapping providers is one file — every call already goes through a single function.

+ • **The vectoriser** NOT A MODEL WE TRAIN [scikit-learn TfidfVectorizer](#) → 384 dimensions

WHAT IT DOES

Turns the user's past replies into numbers so we can find the ones most like the current situation.

NOW VS PRODUCTION

Now: TF-IDF word-frequency vectors, no download, works immediately. **Production:** a small sentence encoder (~22M) for genuine meaning-matching. One-line swap; the interface is identical.

WHERE

Stage 19. This is **the mechanism that makes writing style adapt instantly** — no weights, so nothing to retrain.

STORED

Vectors in SQLite now, **pgvector in Postgres** in production. Per person. Thousands of vectors per user, a few searches a day — nowhere near needing a separate vector database.

Stated honestly: TF-IDF matches shared words, not meaning. It is good enough to prove the shape of the system. A real sentence encoder is needed before we claim semantic matching to anybody.

PRODUCTION

Where each model runs, and what it costs

No GPU appears anywhere in the serving path. A GPU is needed for a few minutes to train, and Kaggle currently gives that away free.

MODEL	TRAINED WHERE	RUNS WHERE	LOADED HOW	SCALES BY
1 Reader	One GPU, once, offline	CPU worker	In memory per worker, ONNX int8	Adding workers
2 Decider	CPU, per user, weekly, as a background job	CPU worker	Per-user file, cached	Adding workers
3 Name finder	One GPU, once, offline	CPU worker	In memory per worker, ONNX int8	Adding workers
4 Writer	Not ours	Vendor API	HTTPS, retry + 4 fallbacks	Vendor quota

How many users each model can carry

Measured speed per email, turned into capacity. A "user" here is 100 emails a day.

MODEL	RUNS ON	SPEED	EMAILS / SECOND ONE CPU CORE	EMAILS / DAY ONE CPU CORE	USERS PER CORE
1 Reader xlm-roberta- base	20% of mail	110 ms	9	785,455	39,273
after ONNX int8	20%	25 ms	40	3,456,000	172,800
2 Decider LightGBM	20% of mail	1 ms	1,000	86,400,000	4,320,000
3 Name finder xlm-roberta- base	8% of mail	120 ms	8	720,000	90,000
after ONNX int8	8%	30 ms	33	2,880,000	360,000
4 Writer gemini-flash- latest	8% of mail	~2 s	Not our hardware. Capacity is the vendor's rate limit, not a core count — and it is the one thing that costs money per call.		

All three of our models sharing one ordinary 4-core server

As built today: ~31,000 users. After ONNX int8 quantisation: **~130,000 users.** Both figures already divided by 3.5 for business-hours peaks, so they are what the box actually carries rather than a theoretical average.

The decider is not the constraint and never will be — one core carries **4.3 million users** of it. The reader is the bottleneck at 39,000 users per core, which is exactly why quantising it is the first optimisation worth doing: it is **4× for one afternoon's work**, and the code is already written.

Compute, calculated

USERS	EMAILS / DAY	REACH STAGE 8	PAID CALLS / DAY	MODEL CPU CORES AS BUILT	MODEL CPU CORES WITH ONNX INT8	PER-USER TRAINING CORES, WEEKLY
1,000	100,000	20,000	8,000	0.04	0.01	0.00
10,000	1,000,000	200,000	80,000	0.37	0.09	0.03
100,000	10,000,000	2,000,000	800,000	3.66	0.86	0.33
1,000,000	100,000,000	20,000,000	8,000,000	36.57	8.56	3.31

Assumptions, stated so they can be challenged: 100 emails per user per day; 20% survive stages 3 and 4 to reach stage 8 (**MEASURED** on this mailbox — 24 of 40 machine mail, 15 of 40 never opened); 8% reach the paid step (**ESTIMATED** , not measured, because the test account has too little human correspondence). Cores are **sustained average**. Mail arrives in bursts during business hours, so provision **3–4× this** for peaks, and add capacity for parsing, the queue, the API and the database — realistically 3–5× again overall.

The shape of the bill

Our own models cost **hardware, not per-email fees** — under one core of model inference at 100,000 users after quantisation. That part is nearly free and does not grow with usage in any frightening way.

The real bill is stage 11. At 100,000 users that is **800,000 calls a day, about 280 million tokens a day** (~250 tokens in for a redacted email, ~100 out for a draft). Multiply by the vendor's current published rate for a Flash-class model — **take that number from their pricing page on the day, not from this document**, because it changes and a stale figure in a business case is worse than no figure.

The lever that controls it is not the price. It is the **8%**. Every stage above 11 exists to push that number down, and each point it falls is 100,000 fewer paid calls a day at this scale.

STATED DELIBERATELY

What we do not know yet

QUESTION	STATUS	WHAT WOULD SETTLE IT
Accuracy on a normal working inbox	UNKNOWN	The test account is ~95% notifications. Point it at a working inbox and hand-label a few hundred emails — one day of work, and it lifts the reader and the decider together.
Redaction quality on our own real mail	UNKNOWN	The public score does not transfer. Must be measured before any privacy claim is made publicly.
What fraction really reaches the paid step	CALCULATED	8% is an estimate. It is the single biggest lever on the running cost, so it should be measured before anyone budgets from it.
Throughput under real concurrency	CALCULATED	Load-test. Believing a spreadsheet about throughput is how projects get surprised.
Which writer produces better drafts	UNKNOWN	Blind comparison on real drafts rated by real users — not a public benchmark.

Custodian · OCloud Solutions · target market US, Canada, UK

MEASURED comes from runs on this machine or published research. **CALCULATED** is extrapolated from stated assumptions. **UNKNOWN** is stated as unknown on purpose.